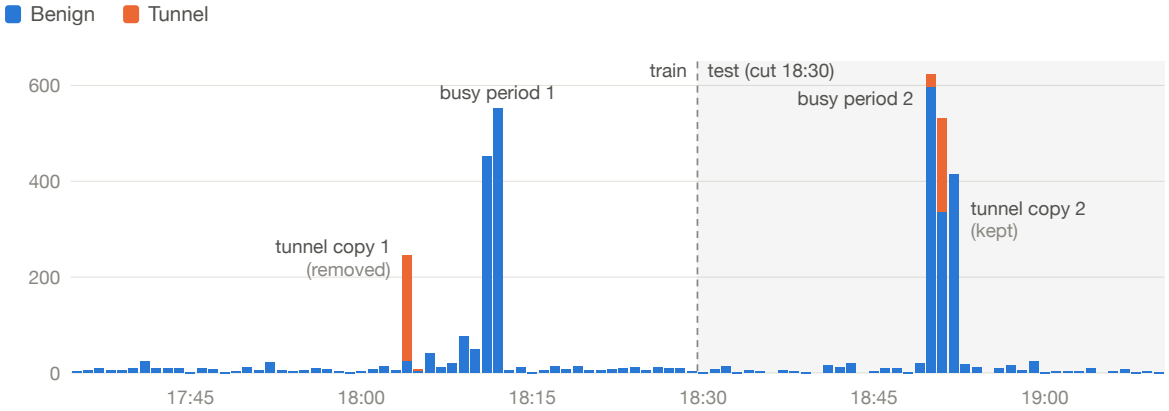
DNS tunnel detection: data pipeline

One capture of normal DNS plus a hidden tunnel session. The pipeline prepares it for an Isolation Forest, and the notebooks record why the feature set looks the way it does. JM2030 Cybersecurity, Group 3, case 1.

DNS queries	Duration	Tunnel session (shown twice)	Default features
3,645	95 min	1	4

The capture, minute by minute



The tunnel session was recorded twice. Copy 2 is an exact replay of copy 1. Benign traffic is quiet, with two busy periods.

How the pipeline runs

dns.log + labels
3,645 rows, joined by row_id

→

prepare.py
Wire-byte names, parent domain, one tunnel copy, split

→

features.py
4 default features per query, per client and domain, causal

→

models
Isolation Forest and baselines (next step)

`src/variants.py` builds stress-test versions of the data (slower tunnel, more benign hosts, a repeated name) that go through the same feature code. 30 acceptance tests cover all three modules.

The setup and why

Keep one tunnel copy
Copy 2 matches copy 1 byte for byte. Keeping both doubles the positives and makes copy 2 look like repeated names.

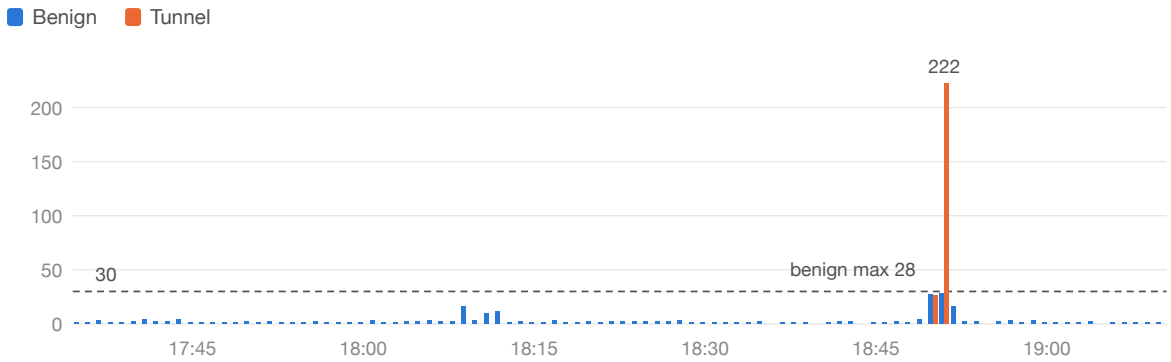
Default keeps copy 2 (18:50)
It sits inside busy period 2, the harder test. Training can then include busy period 1.

Count per client and domain
The context features ask: how many names did this machine ask this domain? Same values here (one benign client); correct for networks with many hosts.

Cut at 18:30
Middle of the quiet gap. Train: 1,592 benign rows, no tunnel. Test: 1,609 benign + 222 tunnel.

Strict variant: keep copy 1, cut at 18:04:36. Training is only 244 quiet rows, so it mostly shows what a load change does.

Why the context features work



Highest `uniq_names_60s` per minute (default run). The session straddles two minute bins, so the first shows 26 although the rolling count reaches 222.

The features

Default: domain context (3)

`uniq_names_60s`
`uniq_ratio_60s`
`norepeat_tol`

Caching forces a new name per query. A 60 s window plus a long memory that survives a repeated name. Catches the whole session, pings included.

Default: name shape (1)

`name_bytes`

Data travels in the name, so names are long. Catches data and probe queries, not pings. One feature on purpose: the three shape features correlate about 0.8.

Ablation columns (5)

`longest_label` `entropy` redundant with `name_bytes`.
`norepeat_evidence` breaks on one repeated name.
`log_cum_uniq` drifts with time.
`data_qtype` (NULL or TXT) is iodine specific.

Left out on purpose: IPs, ports, flags, TTL, EDNS0, rtt, NXDOMAIN, raw volume, domain names. They separate the classes only because of the lab.

All features at a glance

Every feature column in `features.csv`. Median (max) is over the whole default run: benign against tunnel. Row = the characteristic in report Table 2.5 that the feature targets. "Per pair" means counted per client and parent domain, using only queries up to and including the current one.

Feature	Row	What it is exactly	Benign / tunnel
DEFAULT SET (`feature_sets(60)["forced"]`)			
`name_bytes`	2	**Length of the full query name in bytes as sent on the wire (Zeek's `\xNN` escapes count as one byte).**	**20 (103) / 19 (252)**
`uniq_names_60s`	1	**Distinct names this client asked this parent domain in the last 60 s, current query included. Per pair.**	**1 (28) / 111.5 (222)**
`uniq_ratio_60s`	1	**`uniq_names_60s` divided by the number of queries in that same window. 1.0 means no name was repeated in the window.**	**0.40 (1.0) / 1.0 (1.0)**
`norepeat_tol`	1	**`log(1 + d) × d / n`, with `d` distinct names and `n` queries so far. Long memory; one repeated name barely changes it. Per pair.**	**0.40 (2.05) / 4.72 (5.41)**
ABLATION COLUMNS (`extra`), NOT IN THE DEFAULT			
`longest_label`	2	Length in bytes of the longest label in the name (DNS limit 63). Correlates 0.80 with `name_bytes`.	9 (57) / 8 (62)
`entropy`	3	Shannon entropy in bits per character of the subdomain, dots included; 0 if there is none. Correlates 0.80 with `name_bytes`.	2.50 (4.53) / 2.75 (6.30)

Feature	Row	What it is exactly	Benign / tunnel
`norepeat_evidence`	1	`log(1 + d)` while the pair has never asked the same name twice, else 0. Switched off for good by one repeat. Per pair.	0 (1.61) / 4.72 (5.41)
`log_cum_uniq`	1	`log(1 + d)` : distinct names the pair has asked so far. Grows with time since the capture started, so it drifts. Per pair.	1.39 (3.58) / 4.72 (5.41)
OPTIONAL			
`data_qtype`	5	1 if the record type is NULL (10) or TXT (16), else 0. Part of `full` only. NULL is iodine's choice.	0 (0) / 1 (1)

All features are numeric and raw (not scaled). These columns are metadata, never features: `ts` `label` `y` `split` `burst` `is_replay` `phase` `parent` `client` . Full definitions: `docs/data_dictionary.md` .

Why this feature set

Isolation Forest fitted on benign training rows, PR AUC on the test rows (mean of 10 seeds, threshold-free). Primary = tunnel copy 2; strict = copy 1 with only 244 training rows. One tunnel event, so read the direction, not the decimals.

Feature set	Primary	Strict
Context only	0.993	0.985
Default: `name_bytes` + context	**0.993**	**0.974**
Default + `entropy`	0.993	0.870
Default + `longest_label`	0.993	0.809
All three shape features + context	0.975	0.658
Default without `norepeat_tol`	0.969	0.533
Default + `norepeat_evidence`	0.997	0.991
Default + `log_cum_uniq`	0.991	0.887

More shape features rank worse because they are redundant and take splits from the context features. `norepeat_tol` carries the ranking. `norepeat_evidence` scores a little higher on clean data but one repeated name switches it off (see below), so it is an ablation. Details and figures: `notebooks/03_features.ipynb` .

Stress tests (simulated, built from this one capture)

Slower tunnel

Up to 60 times slower (about 25 minutes). ROC AUC stays at 0.99 or higher; the default holds up better than context only at the slowest setting.

More benign hosts

1 to 20 hosts. Default PR AUC 0.986 to 0.993; three shape features fall from 0.98 to 0.70. Per-client counting showed no benefit here: a design choice, not a result.

One repeated name

`norepeat_evidence` PR AUC 0.997 to 0.956. `norepeat_tol` stays at 0.993.

Running it

```
python main.py                      default: copy 2, cut 18:30, 60 s window
python main.py --all --tests        default + strict variant, then the checks
python main.py --window 60 300 900  several window sizes (slow tunnels)
python main.py --session first      only copy 1
```

`data/processed/` is not in git: run `python main.py` first (needs `data/raw/dns.log` from Canvas). Then start with `features.csv`. Columns: `split` (train/test), `y` (label), 4 default features plus 5 ablation columns. Features are raw, not scaled. Default set: `feature_sets(60)["forced"]`.

Suggested modelling

1. Fit Isolation Forest on the default features, training rows only.
2. Choose the threshold without the test rows: a percentile on a benign validation slice. Tukey on the training scores is poorly calibrated here (about 170 benign rows flagged in the primary run), so treat it as a baseline.
3. Evaluate on test: precision, recall, F1, recall per phase, event-level metrics (time to first alert), and look at the false positives.
4. Compare with simple baselines (a rule on `uniq_names_60s` is strong) and one alternative model. Then the ablations, window sizes, other cuts and the variants.

Limits to state

- One tunnel session, one tool, one benign client. The simulations are built from this one capture and show robustness, not generalisation.
- The tunnel is loud, so results will look strong. Slow and noisy variants are the real test.
- The first 28 tunnel queries (12%) stay within the benign range of `uniq_names_60s`. 6.5% of rows are tunnel, so precision is optimistic.
- The feature set is a starting point: `name_bytes` as the only shape feature rests on redundancy, and the numbers are mixed.

Updated 2026-11-12 from data/processed and notebooks 02 and 03. See docs/phase2 - feature engineering overview.md (handoff) and docs/data_dictionary.md for details.